

C++&Qt Day04 hw2 보고서

작성자: Doyeon Park

작성일: 2026.09.08

목차

1. UDP 란?
2. UDP 동작방식
3. UDP Header 구조
4. IP 와 Port
5. UDP 의 특징
6. TCP 와 UDP 비교
7. UDP 의 장점과 단점
8. UDP 가 사용되는 경우
9. UDP Checksum
10. 데이터 손실, 순서변경, 중복
11. UDP Datagram 크기와 Fragmentation
12. UDP 와 Congestion Control
13. QT 에서 UDP 사용
14. Qt UDP 송수신 과정
15. UDP 를 공부하면서 알게 된 점

1. UDP 란?

UDP는 User Datagram Protocol의 약자로, 인터넷에서 데이터를 전달할 때 사용하는 전송 계층(Transport Layer) 프로토콜 중 하나이다.

인터넷 통신에서는 일반적으로 응용 프로그램이 바로 다른 컴퓨터와 통신하는 것이 아니라 여러 계층을 거쳐 데이터를 전달하는데, 대략 다음과 같이 볼 수 있다.

응용 계층(Application) → 전송 계층(TCP/UDP) → 인터넷 계층(IP) → 네트워크 (Ethernet / Wifi)

UDP는 이 중에서 전송 계층에 속하며 IP를 이용하여 데이터를 목적지까지 전달하는 역할을 한다.

UDP에서 전달되는 하나의 데이터 단위를 Datagram이라 한다. UDP는 데이터를 하나하나 독립적인 데이터그램 단위로 전달한다.

UDP의 표준을 정의한 RFC768에서는 UDP가 메시지를 전달하는 최소한의 전송 기능을 제공하며, 데이터의 전달이나 중복방지를 보장하지 않는다고 정의한다. 또한 순서가 보장되는 신뢰성있는 데이터가 필요한 경우 TCP를 사용해야 한다고 한다.

OSI 7 Layer

L7	응용 계층 (Application Layer)
L6	표현 계층 (Presentation Layer)
L5	세션 계층 (Session Layer)
L4	전송 계층 (Transport Layer)
L3	네트워크 계층 (Network Layer)
L2	데이터 링크 계층 (Data Link Layer)
L1	물리 계층 (Physical Layer)

TCP/IP 4 Layer

L4	응용 계층 (Application Layer)
L3	전송 계층 (Transport Layer)
L2	인터넷 계층 (Internet Layer)
L1	네트워크 액세스 (Network Access Layer)

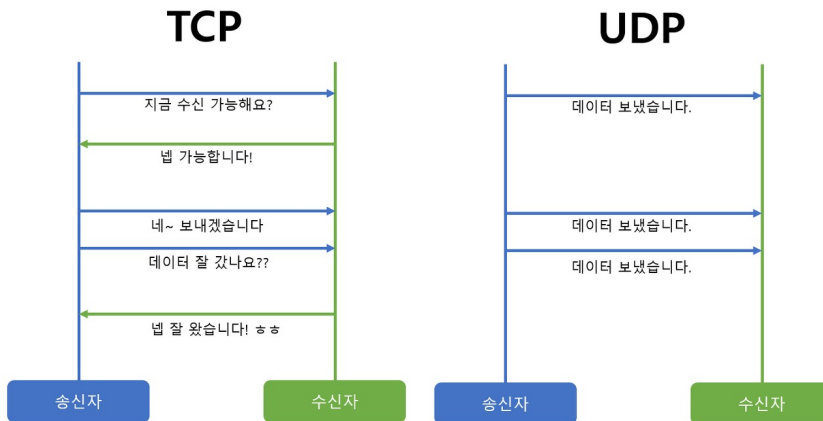
이번 과제에서 천지인 키보드와 Qt UI는 응용 프로그램에 해당하며, QUdpSocket을 이용한 메시지 전송은 UDP 전송 계층을 이용한다고 볼 수 있다.

2. UDP의 동작 방식

UDP의 가장 큰 특징 중 하나는 Connectionless, 즉 비연결형 통신이라는 점이다.

TCP에서는 데이터를 보내기 전에 송신자와 수신자 사이에 연결을 형성하는 과정이 필요한 반면, UDP는 이러한 연결 설정 과정 없이 목적지 IP 주소와 Port 번호를 지정하면 바로 데이터그램을 보낼 수 있도록 되어있다.

개념적으로 보면 다음과 같다.



UDP에서는 데이터그램 하나가 다른 데이터그램과 독립적으로 처리된다.

따라서 송신자는 데이터그램을 보낸 뒤 상대방이 실제로 받았는지를 UDP 자체에서는 확인하지 않는다.

이러한 구조때문에 프로토콜 처리가 비교적 단순하긴 하지만, 데이터가 반드시 전달되어야 하는 상황에선 UDP 방식은 불리할 수 있다.

2.1 UDP 설정 영역

UDP 통신을 위해 내 Port, 상대 Port, 상대 IP, 수신시작버튼으로 구성하였다.

내 Port 는 현재 프로그램이 UDP 데이터를 받을 포트 번호이고, 상대 Port 는 메시지를 전송할 상대 프로그램의 포트 번호이다. 상대 IP 에는 메시지를 전송할 상대 컴퓨터의 IP 주소를 입력하고 수신시작 버튼을 누르면 입력한 내 Port 에 bind()하여 UDP 데이터를 받을 준비를 한다.

The screenshot shows a window titled 'Type Here' with a dotted background. It contains four input fields: '내 Port :', '상대 Port :', '상대 IP :', and 'Send Message :'. A '수신시작' (Start Receiving) button is located next to the '내 Port' field.

2.2 채팅창

채팅 내용은 QTextEdit 을 이용하여 표시하였다.

채팅 기록을 사용자가 직접 수정할 필요가 없기 때문에 readOnly 상태로 설정하였다.

The screenshot shows a chat window with a 'Send Message :' input field at the top. Below it is a large text area containing the chat history: '나: 안녕하세요', '상대: 만나서 반갑습니다ㅇㅇ', and '상대: 만나서 반갑습니다'.

3. UDP Header 구조

UDP 는 TCP 에 비해 Header 구조가 매우 단순하다.

UDP Header 는 총 8 Byte 이며 다음 4 개의 필드로 이루어진다.

0	16	31 비트
발신 포트번호	수신 포트번호	
(패킷전체) 길이	(패킷전체) 체크섬	
... 데이터 ...		

UDP Header 의 네 필드는 모두 16bit 이다. 따라서 UDP Header 자체는 $16\text{bit} \times 4 = 64\text{ bit}$, 즉 8Byte 이다. Length 필드는 UDP Header 와 데이터를 모두 포함한 데이터그램 전체 길이를 나타내므로 최소값은 Header 크기인 8Byte 이다.

3.1 Source Port

데이터를 보낸 프로그램의 Port 번호로, 예를들어 이번과제에서 프로그램이 다음과 같이 설정되어 있다고 가정하자.

내 Port : 10004

이 경우 10004 는 현재 프로그램이 UDP 데이터를 수신하기 위해 사용하는 Port 번호이다.

3.2 Destination Port

데이터를 받을 프로그램의 Port 번호이다.

이번 과제의 UI 에서

상대 Port : 10004

와 같이 입력한 값이 Destination Port 에 해당한다.

3.3 Length

UDP Header 와 데이터 전체의 길이를 나타낸다.

즉, $\text{UDP Length} = \text{UDP Header} + \text{실제 데이터}$ 이다.

3.4 Checksum

전송 과정에서 데이터에 오류가 발생했는지 검사하기 위한 값이다.

이에 대해선 뒤에서 자세히 설명하겠다.

4. IP 와 Port

IP 주소는 네트워크에서 어느 컴퓨터로 보낼지 결정한다.

Port 번호는 해당 컴퓨터에서 어느 프로그램으로 데이터를 전달할지 결정한다.

예를들어 172.100.0.183:10004 라 하면

172.100.0.183 → 상대 컴퓨터

1004 → 상대 컴퓨터의 UDP 프로그램

을 의미한다.

5. UDP 의 특징

UDP 의 주요 특징을 정리하면 다음과 같다.

특징	내용
비연결형	데이터 전송 전에 연결 설정을 하지 않음
Datagram 방식	메시지 하나하나를 독립된 데이터그램으로 전달
전달 보장 없음	데이터가 반드시 도착한다는 보장이 없음
순서 보장 없음	보낸 순서와 받은 순서가 다를 수 있음
중복 방지 없음	같은 데이터그램이 중복 도착할 가능성이 있음
단순한 Header	UDP Header 는 8 Byte
낮은 프로토콜 오버헤드	TCP 보다 제어 과정이 단순함

특히 UDP 에서는 TCP 와 달리 재전송, 순서복원, 중복 제거 기능을 기본적으로 제공하지 않기 때문에 이 같은 기능이 필요할 경우 사용자가 직접 프로그래밍 해야한다는 특징이 있다.

6. TCP 와 UDP 비교

구분		TCP	UDP
연결 방식	연결 지향		비연결
데이터 형식	Byte Stream		Datagram
전달 보장	있음		없음
순서 보장	있음		없음
재전송	있음		없음
Header	상대적으로 큼		8 Byte
제어 과정	많음		적음
주요 목적	신뢰성		단순성·실시간성

구분	TCP	UDP
예시	웹 전송, 파일 전송 등	실시간 통신, DNS 등

TCP에서는 데이터를 정확하게 보내는 것이 중요한 경우에 알맞다.

예를 들어 파일을 다운로드 하는과정에서 데이터 일부가 사라지면 치명적이기 때문에 재전송과 순서 보장이 중요하다. 이러한 경우 TCP가 적합하다 볼 수 있다.

반대로 실시간 음성이나 게임 데이터에서는 과거의 데이터 하나를 다시 전송하기 위해 기다리는 것보다 최신 데이터를 빠르게 처리하는 것이 중요할 때가 있다. 이러한 경우에는 UDP가 적합하다 볼 수 있다.

따라서 TCP와 UDP의 차이를 단순히 좋다 나쁘다로 보는 것이 아닌 각 프로그램의 목적에 따라 적절한 프로토콜을 선택해야 한다.

7. UDP의 장점과 단점

7.1 장점

UDP는 연결 설정 과정이 없고 Header가 단순하다.

따라서 애플리케이션이 필요로 하지 않는 기능에 대한 부담을 줄일 수 있다.

또한 데이터 그램단위로 통신하기 때문에 하나의 메시지를 독립적으로 처리하기 쉽다.

UDP의 이러한 특성 때문에 빠른 반응이 필요한 프로그램에서 유리하게 사용될 수 있다.

7.2 단점

UDP 자체에서는 다음 기능을 제공하지 않는다.

전송 성공 확인
재전송
순서복구
중복제거
혼잡 제어

따라서 이러한 기능이 필요하다면 프로그램을 작성하는 사용자가 별도로 직접 구현해야 한다.

예를 들어 매우 중요한 제어 명령을 UDP로 보낸다 할때, 다음과 같이 설계할 수 있다.

명령 전송 → 응답 기다림 → 일정 시간 응답 없음 → 재전송

즉 UDP가 빠르다는 것보다 UDP 자체가 제공하는 기능이 적기 때문에 좀더 응용 프로그램이 필요한 기능을 좀 더 다양하게 선택해서 구현할 수 있다고 이해하는것이 좋다.

8. UDP 가 사용되는 경우

UDP 는 특히 실시간성이 중요하거나 작은 데이터를 반복적으로 전달하는 환경에서 많이 사용된다.

대표적인 예로 다음을 들 수 있다.

DNS 질의
실시간 음성 통신
영상 스트리밍 일부
온라인 게임
센서 데이터
로봇 상태 데이터

그중 하나를 예를 들자면, 로봇이 주기적으로 자신의 현재 위치를 보내는 상황을 생각해볼 수 있다.

10:00:00.00 → 위치 1
10:00:00.10 → 위치 2
10:00:00.20 → 위치 3

위치 1 데이터가 네트워크에서 손실되었는데 이미 위치 3 까지 시간이 지났다면, 오래된 위치 데이터를 다시 받는 것보다 최신 위치를 받는 것이 더 중요할 수 있다.

이러한 특징때문에 로봇이나 실시간 제어 시스템에서 UDP 를 사용하게 되는 경우가 많다고 한다.

9. UDP Checksum

UDP Header 에는 Checksum 이라는 필드가 있다.

Checksum 의 목적은 네트워크를 통해 데이터를 전달하는 과정에서 데이터가 손상되었는지를 검사하는 것이다.

RFC768 의 UDP checksum 계산에는 UDP Header 와 데이터뿐만 아니라 IP 계층의 일부 정보를 이용한 Pseudo Header 도 사용된다. 여기에는 Source Address, Destination Address, Protocol, UDP Length 등이 포함된다.

즉, IP 정보 일부 + UDP Header + UDP Data → Checksum 계산 이다.

IPv4 의 초기 UDP 정의 에서는 Checksum 을 사용하지 않는 표현도 허용했었지만, UDP Checksum 생성 및 검증 기능을 구현해야 하고 기본적으로 Checksum 을 사용해야한다.

중요한 것은 Checksum 이 데이터를 복구해 주는 기능은 아니다.

Checksum 은 단순히 오류검출만 하는것이고 UDP 자체에는 재전송기능이 없기 때문이다.

하지만 이는 프로그래밍으로 보완가능하다.

10. 데이터 손실, 순서변경, 중복

UDP 를 사용할 때 반드시 알고 있어야 하는 특징이다.

UDP 에서 송신자가:

A
B
C
D

순서로 데이터를 보냈다고 해도 반드시 수신자가:

A
B
C
D

로 받는다는 보장은 없다.

예를 들어:

A
C
B
D

처럼 순서가 변경될 수도 있으며,

A
B
D

와 같이 C 가 손실될 수도 있다.

또는 드물게

A
B
B
C
D

처럼 데이터그램이 중복될 가능성도 있다.

따라서 UDP 를 사용하는 프로그램이 중복된 데이터그램을 처리하고, 순서가 필요한 프로그램이라면 프로그램 코드 자체에서 순서를 구분하도록 기능을 넣어야 한다.

순서 변경 오류를 해결하는 방법으로 데이터에 번호를 추가하는 방법이 있다.

1 : Hello
2 : Robot

3: UDP

이렇게 설정하면 번호를 확인하여 순서를 확인할 수 있게 된다.

이번 과제에선 이러한 디버그 요소를 넣지는 않았지만 UDP는 통신에서 기본적인 것만 제공하고 사용자가 예외처리 및 디버그를 직접 할 수 있다는 것이 흥미로운 점인 것 같다.

11. UDP Datagram 크기와 Fragmentation

UDP에서는 하나의 메시지를 하나의 데이터그램으로 전송한다.

하지만 매우 큰 데이터그램을 보낸다고 해서 항상 그대로 네트워크를 통과하는 것은 아니다.

네트워크에는 한 번에 전달할 수 있는 패킷 크기에 제한이 있으며, 데이터가 너무 크면 IP 계층에서 Fragmentation(단편화)이 발생할 수 있다.

단편화의 문제점은 Fragment 중 하나라도 손실되면 원래 데이터그램 전체를 정상적으로 사용할 수 없게 될 수도 있다는 것이다.

따라서 매우 큰 데이터그램은 IP 계층에서 fragmentation 될 가능성이 있으므로 크기가 큰 데이터를 전송할 때 사용자가 프로그래밍을 통해 별도의 안전장치를 구현할 필요가 있다.

이번 과제에서는 큰 데이터를 주고 받지 않기 때문에 이 문제를 고려할 필요는 없지만, 대용량 데이터를 UDP로 직접 보내는 경우에는 도의 데이터 분할 방법을 설계할 필요가 있다.

12. UDP 와 Congestion Control

UDP에는 TCP와 같은 내장된 혼잡 제어 기능(Congestion Control)이 없기 때문에 네트워크에 너무 많은 데이터가 동시에 전달되면 처리할 수 있는 용량을 넘겨버릴 수 있다.

송신량 증가 → 네트워크 용량 초과 → Queue 증가 → Packet Loss 증가

TCP는 네트워크 상황에 따라 전송량을 조절하는 기능을 가지고 있지만 UDP는 이 기능을 갖고 있지 않다.

따라서 인터넷 환경에서 UDP를 사용하는 프로그램은 필요한 경우 프로그램 코딩 단계에서 데이터 전송 속도를 조절해야 할 수도 있다.

따라서 UDP 자체에 congestion control이 없기 때문에 UDP 기반 응용 프로그램이 네트워크 혼잡을 악화시키지 않도록 프로그래밍 단계에서 적절한 조치를 취해야 한다.

13. QT 에서 UDP 사용

Qt에서는 UDP 통신을 위해 QUdpSocket 클래스를 사용할 수 있다.

일반적인 사용 방법은 다음과 같다.

QudpSocket 생성 → bind() → readyRead 발생 → receiveDatagram()

송신할때는 다음과 같은 방식을 사용한다.

QbyteArray 데이터 → writeDatagram() → 상대 IP + 상대 Port

방식을 사용한다.

수신:

```
udpSocket → bind(  
    QhostAddress::AnyIPv4,  
    localPort  
);
```

bind()는 소켓을 로컬 Port 에 연결하여 해당 Port 로 들어오는 데이터를 받을 수 있도록 한다.

그리고

```
connect(  
    udpSocket,  
    &QUdpSocket::readyRead,  
    this,  
    &MainWindow::udpRead  
);
```

를 이용하여 데이터가 도착하면 udpRead()가 실행되도록 한다.

14. Qt UDP 송수신 과정

송신:

- 천지인 키보드
- committedText
- QString
- toUtf8()
- QByteArray
- writeDatagram()
- 상대 IP + Port

수신:

- UDP Datagram 도착
- readyRead
- udpRead()
- hasPendingDatagrams()

- receiveDatagram()
- QByteArray
- QString::fromUtf8()
- > 채팅창 출력

Qt 의 receiveDatagram()은 데이터뿐 아니라 송신자의 주소와 Port 정보까지 포함하는 QNetworkDatagram 을 반환할 수 있다.

15. UDP 를 공부하면서 알게 된 점

UDP 는 데이터를 보내기 전에 상대방과 연결을 형성하지 않고, 목적지의 IP 주소와 Port 번호를 지정하여 데이터그램을 바로 전송한다. 이 때문에 구조가 단순하고 별도의 연결 과정이 필요하지 않지만, 데이터가 실제로 상대방에게 도착했는지 확인하거나 손실된 데이터를 자동으로 다시 전송하지 않는다는 특징이 있다는 것을 알게 되었다.

뿐만아니라 UDP 는 데이터의 순서를 보장하지 않기 때문에 여러 개의 데이터그램을 연속으로 보낼 경우 송신 순서와 수신 순서가 달라질 가능성도 있다는 것을 알게 되었다. 이번 과제에서는 짧은 문자 메시지를 주고받는 프로그램이었기 때문에 이러한 문제를 직접 경험하지는 않았지만, 실제로 UDP 를 이용하여 중요한 데이터를 전송한다면 데이터 번호를 추가하거나 수신 확인 기능을 별도로 구현해야 할 수도 있다는 점을 알게 되었다.

또한 처음에는 IP 와 Port 모두 통신을 위해 사용하는 숫자 정도로만 생각하였다. 그러나 실제 두 대의 컴퓨터에서 프로그램을 실행하여 통신해 보면서 IP 는 네트워크상에서 통신할 컴퓨터를 찾기 위한 주소이고, Port 는 해당 컴퓨터 안에서 어떤 프로그램으로 데이터를 전달할지를 구분하기 위한 번호라는 것을 이해하게 되었다.

Qt 에서는 QUdpSocket 클래스를 이용하여 UDP 통신을 구현하였다. 처음에는 네트워크 통신을 구현하려면 복잡한 반복문이나 별도의 통신 처리 과정이 필요할 것이라고 생각했지만, Qt 에서는 Signal 과 Slot 기능을 이용하여 비교적 간단하게 데이터를 받을 수 있었다.

bind()를 이용하여 현재 프로그램이 사용할 Port 를 열고, UDP 데이터가 들어오면 readyRead Signal 이 발생하도록 하였다. 이 Signal 을 udpRead() 함수와 연결하여 데이터가 도착했을 때만 수신 함수가 실행되도록 구현하였다.

이를 통해 이전에 버튼 클릭이나 Timer 에서 사용했던 Signal / Slot 이 단순한 UI 기능뿐만 아니라 네트워크 통신에서도 사용될 수 있다는 것을 알게 되었다.

또한 데이터를 보낼 때는 QString 형태로 저장되어 있던 메시지를 toUtf8()을 이용하여 QByteArray 로 변환하고, writeDatagram()을 이용하여 상대방의 IP 와 Port 로 전송하였다. 반대로 데이터를 받을 때는 receiveDatagram()으로 받은 데이터를 다시 QString::fromUtf8()을 이용하여 문자열로 변환한 뒤 채팅창에 출력하였다.

이 과정을 통해 화면에 보이는 문자열이 그대로 네트워크로 전송되는 것이 아니라, 실제 전송을 위해 바이트 형태의 데이터로 변환하는 과정이 필요하다는 것도 알게 되었다.

이번 과제를 통해 프로그램을 작성할 때 오류가 발생하면 무조건 코드만 확인하는 것이 아니라 빌드 시스템, 파일 경로, 네트워크 설정, IP 와 Port 값 등 프로그램 외부의 설정도 함께 확인해야 한다는 점을 배울 수 있었다.

최종적으로 한 컴퓨터에서 프로그램 두 개를 실행하여 먼저 UDP 송수신을 테스트한 뒤, 실제 두 대의 컴퓨터를 이용하여 메시지를 주고받는 데 성공하였다. 처음에는 UDP 의 개념이 잘 와닿지 않았지만, 직접 IP 와 Port 를 설정하고 메시지를 송수신하면서 UDP 의 동작 과정을 실제 프로그램과 연결하여 이해할 수 있었던 좋은 경험이었던 것 같다.